OPERATING SYSTEMS ASSIGNMENT 3 – CO3

CO3 Coverage: Evaluate memory-management mechanisms such as linking, paging and segmentation to determine their effectiveness for different system requirements – linking and loading, paging, segmentation, virtual memory, demand paging, page-replacement techniques and disk management.

Nature of Assignment: Individual
Duration: 1 Week
Total Marks: 50
Submission Date: _____

Bloom's Levels Assessed: K5 – Evaluate (every task in this assignment is at K5 level)

Mandatory Execution and GitHub Submission

- Every student must individually execute every task in this assignment, including all "Complete the Code" programs.
- Create a GitHub repository named CSA04-<RegisterNumber>-Assignment3 and share it with the course faculty (public, or private with the faculty added as a collaborator).
- Keep one folder for each task (for example: Task01, Task02 … or Q1, Q2 …), containing the C source files, shell scripts, configuration files (Dockerfile, YAML), command outputs and screenshots.
- Include a README.md with your name, register number, compiler and version (for example gcc 13), the Linux distribution and version used, the exact gcc command used to compile (for example gcc -std=c11 -Wall -Wextra -pthread), the steps to run each program, and a short note of the result of each task.
- Commit your work progressively – at least one commit for each task. The commit history will be checked; a single upload of all files on the last day will not be accepted.
- Submit the GitHub link below and on the cover page of your report. An assignment submitted without a working GitHub link will not be evaluated.

GitHub Repository Link: https://github.com/192511225simats-code/operating-system.git

How to Answer the Additional Questions

Complete the Code: Copy the given code into your repository, replace every TODO with working code without changing the rest of the program, execute it and compare your output (or result) with the expected output.

Alternative Concept and Complexity Analysis: Describe how the solution would change if the alternative concept were used, with code or pseudocode, and analyse and compare the time complexity and space complexity of both solutions using Big-O notation, supported by your own measurements wherever possible.

Assignment Principle

Every question asks you to design, implement, analyse and evaluate the solution yourself, so that you demonstrate evidence of:

Design → Implementation → Analysis → Evaluation

Q1 – Linking, Loading and Address Binding 6 Marks

Bloom's Level: K5 – Evaluate

Problem Context – Updating a Banking Application

A bank's branch software is built from many modules and libraries. When a security fix is made to the encryption library, every branch must currently receive a new 200 MB executable.

Implementation Tasks You must:

- Build the same program with static linking and with dynamic (shared) libraries
- Compare the sizes of the executables and list the shared libraries with ldd
- Load a library at run time with dlopen() and dlsym()
- Show the addresses at which the program and its libraries are loaded (/proc/self/maps)

Evaluation You must:

- Explain compile-time, load-time and execution-time address binding
- Explain the roles of the linker and the loader
- Evaluate static and dynamic linking for memory use, start-up time and ease of updating • Explain position-independent code

Experimental Investigation You must:

- Measure the start-up time and memory use of both versions
- Replace the shared library without recompiling the program and show the change
- Run two copies and show which pages are shared
- Test what happens when the shared library is missing

Conclusion

Which linking method should the bank use, and why?

Alternative Concept and Complexity Analysis

If the bank loaded the encryption library at run time with dlopen() only when needed, instead of linking it dynamically at start-up, what would the solution be? Evaluate the start-up time, the memory used when encryption is not needed, and the error handling needed if the library is missing.

Q2 – Paging 8 Marks

Bloom's Level: K5 – Evaluate

Problem Context – Memory for a Multi-User College Server

A college server runs hundreds of student programs of very different sizes. Allocating each program one contiguous block of memory left many small unusable gaps, and large programs could not be loaded although enough total memory was free.

Implementation Tasks You must:

- Explain external fragmentation using a diagram of the contiguous allocation

- Translate logical addresses to physical addresses with a page table
- Calculate the size of the page table for a 32-bit and a 48-bit address space with 4 KB pages • Calculate the effective memory-access time with and without a TLB

Evaluation You must:

- Explain how paging removes external fragmentation and why internal fragmentation remains
- Evaluate page sizes of 1 KB, 4 KB and 2 MB
- Explain hierarchical (multi-level), hashed and inverted page tables
- Explain how protection and shared pages are provided with paging

Experimental Investigation You must:

- Calculate the internal fragmentation for 100 processes of random sizes with each page size
- Calculate the effective access time for TLB hit ratios of 80%, 95% and 99%
- Find the page size of your Linux system (getconf PAGESIZE) and the page faults of a program (/usr/bin/time -v)
- Test addresses at the start and end of a page and beyond the last page

Conclusion

What page size would you choose for the college server, and why?

Complete the Code

Complete TODO 1 below and execute the program.

```
#include <stdio.h>


#define PAGE_SIZE 1024
#define PAGES 8


/* Page table of a process: frame number, or -1 if the page is not in memory. */ int page_table[PAGES] = {5, 2, -1, 7, 0, -1, 3, -1};


int main(void)
{
   unsigned logical[] = {0, 1500, 3100, 6200, 9000, 2049};     for (int i = 0; i < 6; i++) {
unsigned a = logical[i];
      // TODO 1: Split a into page number p = a / PAGE_SIZE and offset d = a %
      //   PAGE_SIZE. If p is outside the table print "invalid address"; if the frame      //   is -1 print "page fault"; otherwise print the
physical address frame *          //   PAGE_SIZE + d


   }
   return 0;
}
```

Expected output:

```
   0 -> page 0 offset    0 : physical 5120
1500 -> page 1 offset  476 : physical 2524
3100 -> page 3 offset   28 : physical 7196
6200 -> page 6 offset   56 : physical 3128
9000 -> page 8 offset  808 : invalid address
2049 -> page 2 offset    1 : page fault
```

Change PAGE_SIZE to 2048 and recalculate the results by hand before running the program again.

Alternative Concept and Complexity Analysis

If the server used an inverted page table (one entry per physical frame) instead of a per-process multi-level page table, what would the solution be? Evaluate the space used (proportional to physical memory versus to the address space of each process) and the time to translate an address (a hash look-up versus a walk of several levels).

Q3 – Segmentation 6 Marks

Bloom's Level: K5 – Evaluate

Problem Context – Student Programs with Separate Code, Stack and Data

Students' programs have code that should be read-only and shared by everyone running the same compiler, a stack that grows, and data of varying size. Paging alone treats all of these the same.

Implementation Tasks You must:

- Translate (segment, offset) addresses using a segment table with base and limit
- Show what happens when an offset exceeds the segment limit
- Draw how segmentation with paging (as in the Intel x86 architecture) translates an address • Show the segments of a running program on Linux (/proc/<pid>/maps)

Evaluation You must:

- Explain how segmentation matches the programmer's view of memory
- Explain how segments are protected and shared
- Evaluate the external fragmentation caused by segmentation
- Compare pure paging, pure segmentation and segmented paging

Experimental Investigation You must:

- Translate 8 addresses by hand and verify them with a small program
- Trigger a segmentation fault and explain what the OS did
- Calculate the fragmentation after loading and removing 20 segments of random sizes • Compare the size of the tables needed in each scheme

Conclusion

Why do modern operating systems rely mainly on paging?

Alternative Concept and Complexity Analysis

If the programs used pure paging instead of segmentation, what would the solution be? Evaluate the fragmentation (internal versus external), the size of the tables, the time to translate an address, and how easily code is shared and protected.

Q4 – Virtual Memory and Demand Paging 8 Marks

Bloom's Level: K5 – Evaluate

Problem Context – Running Large Medical-Imaging Software on a Laptop

A radiologist needs to run an image-processing program that uses 24 GB of memory on a laptop with 16 GB of RAM.

Implementation Tasks You must:

- Explain the steps of handling a page fault
- Calculate the effective access time for page-fault rates of 1 in 1,000, 1 in 100,000 and 1 in 1,000,000 (memory 100 ns, fault service 8 ms)
- Explain copy-on-write and how it makes fork() fast
- Explain thrashing and the working-set model

Evaluation You must:

- Explain how demand paging lets a 24 GB program run in 16 GB
- Evaluate the maximum acceptable page-fault rate for a 10% slow-down
- Explain how the number of frames per process should be decided (equal and proportional allocation)
- Explain how the OS detects and stops thrashing

Experimental Investigation You must:

- Write a program that touches memory in sequential and random order and count its page faults
- Measure the run time as the memory used grows beyond the free RAM (use a virtual machine or a memory limit)
- Show copy-on-write with fork() by measuring memory before and after writing
- Plot the page faults against the number of frames for a reference string

Conclusion

Is buying more RAM or improving the program's locality the better solution for the radiologist? Justify.

Alternative Concept and Complexity Analysis

If the radiologist's program were run on a machine with 32 GB of RAM instead of relying on demand paging, what would the solution be? Evaluate the effective access time in both cases for the measured page-fault rate, and the cost of extra memory against the cost of the slower run.

Q5 – Page-Replacement Techniques 8 Marks

Bloom's Level: K5 – Evaluate

Problem Context – Choosing a Policy for a Web Server

A web server keeps frequently used pages of data in memory. When memory is full, the OS must choose which page to remove, and a poor choice causes slow responses.

Implementation Tasks You must:

- Implement FIFO, LRU and Optimal page replacement
- Count the page faults for a given reference string with 3 and 4 frames
- Demonstrate Belady's anomaly with FIFO
- Explain the clock (second-chance) algorithm as a practical approximation of LRU

Evaluation You must:

- Explain why Optimal is the best possible but cannot be implemented
- Explain why LRU never suffers from Belady's anomaly (the stack property)
- Evaluate the hardware support LRU needs and how the clock algorithm avoids it • Compare the algorithms for a workload with strong locality

Experimental Investigation You must:

- Generate reference strings with strong locality and with random access, and count the faults of each algorithm for 3 to 10 frames
- Plot the faults against the number of frames for each algorithm
- Measure the running time of each algorithm for 100,000 references
- Verify your program's results with a hand calculation for one string

Engineering Decision

Which page-replacement policy would you recommend for the web server? Use your measurements.

Complete the Code

Complete TODO 1 below to finish LRU, and execute the program.

```
#include <stdio.h>


#define MAXF 8


int find(const int f[], int n, int page)
{
   for (int i = 0; i < n; i++) if (f[i] == page) return i;     return -1; }


int fifo(const int ref[], int len, int frames)
{
   int f[MAXF], used = 0, next = 0, faults = 0;     for (int i = 0; i < len; i++)
{        if (find(f, used, ref[i]) >= 0) continue;         faults++;
      if (used < frames) f[used++] = ref[i];         else { f[next] = ref[i]; next = (next + 1) % frames; }
   }      return faults; }


int lru(const int ref[], int len, int frames)
{
   int f[MAXF], last[MAXF], used = 0, faults = 0;     for (int i = 0; i < len;
i++) {         int k = find(f, used, ref[i]);         if (k >= 0) { last[k] = i; continue;
}         faults++;
      // TODO 1: If a frame is free use it; otherwise replace the frame whose last use     //   (last[]) is the oldest. Record last[] = i for the
frame used


   }      return faults; }


int optimal(const int ref[], int len, int frames)
{
   int f[MAXF], used = 0, faults = 0;     for (int i = 0; i < len; i++) {         if (find(f, used, ref[i]) >=
0) continue;         faults++;         if (used < frames) { f[used++] = ref[i]; continue; }         int victim
= 0, farthest = -1;         for
(int j = 0; j < frames; j++) {
         int next = len;                /* never used again */
         for (int t = i + 1; t < len; t++) if (ref[t] == f[j]) { next = t; break; }       if (next > farthest) { farthest = next; victim = j; }
      }         f[victim] = ref[i];
   }      return faults; }


int main(void)
{
   int ref[] = {7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1};     int len = 20;      for (int fr = 3; fr <= 4; fr++)
printf("%d frames: FIFO %d, LRU %d, Optimal %d\n", fr, fifo(ref, len, fr),               lru(ref, len, fr), optimal(ref, len,
fr));     int b[] = {1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5};     printf("Belady string, FIFO: 3 frames %d faults, 4 frames %d
faults\n",           fifo(b, 12, 3), fifo(b, 12, 4));     return 0;
}
```

Expected output:

3 frames: FIFO 15, LRU 12, Optimal 9 4 frames: FIFO 10, LRU 8, Optimal 8 Belady string, FIFO: 3 frames 9 faults, 4 frames 10 faults

Explain why FIFO has more faults with 4 frames than with 3 for the second reference string, and why LRU cannot behave like this.

Alternative Concept and Complexity Analysis

If the clock (second-chance) algorithm were used instead of exact LRU, what would the solution be? Evaluate the time per reference (updating a time stamp or a list on every access versus setting one bit), the extra space per frame, and how close its page faults come to LRU in your simulation.

Q6 – Disk Management 4 Marks

Bloom's Level: K5 – Evaluate

Problem Context – Storage for a Hospital Records Server

The hospital records server must start reliably, survive the failure of a disk and have enough swap space for peak load.

Implementation Tasks You must:

- Explain low-level formatting, partitioning and logical formatting
- Explain the boot block and the boot process
- Explain how bad blocks are handled
- Decide the size and location of the swap space

Evaluation You must:

- Compare RAID 0, 1, 5 and 10 for reliability, capacity and speed
- Choose the RAID level for the records server
- Evaluate swap on a separate disk versus a swap file
- Explain how disk partitions are viewed on Linux (lsblk, fdisk -l)

Experimental Investigation You must:

- Show the partitions and swap of your Linux system
- Calculate the usable capacity of 6 disks of 4 TB at each RAID level
- Calculate the probability of data loss for RAID 1 and RAID 5 given a disk failure rate
- Measure the read speed of a file on SSD and hard disk if both are available

Conclusion

Which storage configuration would you recommend for the records server, and why?

Alternative Concept and Complexity Analysis

If the records server used RAID 6 instead of RAID 5 (or RAID 10), what would the solution be? Evaluate the usable capacity of 6 disks of 4 TB, the time to write a block (the extra parity calculation), and the number of disk failures that can be survived.

Q7 – Design a Memory-Management Plan from First Principles 10 Marks

Bloom's Level: K5 – Evaluate

Problem Context – Memory Management for an Edge Device in a Smart Factory

A smart factory installs edge devices with 2 GB of RAM and no swap disk. Each device runs a realtime control task that must never be delayed by a page fault, a machine-learning task that needs 1.5 GB, and a logging task. The vendor wants to add more tasks in the future.

You are not given the design. You must develop it from the requirements.

Stage 1 – Interpret the Problem | 2 Marks

- Identify the memory needs and timing constraints of each task
- Identify which tasks can tolerate page faults and which cannot
- State the requirements as measurable statements

Stage 2 – Compare Candidate Mechanisms | 3 Marks

- Compare contiguous allocation, paging, segmentation and demand paging for each task
- Decide how memory is locked for the real-time task (for example mlock()) • Present a comparison table with justification

Stage 3 – Implement and Verify | 3 Marks

- Write a program that locks the memory of a time-critical loop and measure its worst-case delay with and without locking
- Simulate the memory use of the three tasks with a page-replacement simulator • Verify that the real-time task is never delayed

Stage 4 – Defend the Design | 2 Marks

- Justify your plan using your measurements
- Explain what changes if a fourth task of 500 MB is added
- Identify one remaining limitation

Alternative Concept and Complexity Analysis

If the edge device used a real-time operating system with no virtual memory instead of Linux with locked memory for the control task, what would the solution be? Evaluate the worst-case delay of the control task, the memory available to the machine-learning task, and the effort of adding new tasks.

MARK DISTRIBUTION

Question	Assessed Area	Bloom's Level	Marks
Q1	Linking, loading and address binding	K5 – Evaluate	6
Q2	Paging	K5 – Evaluate	8
Q3	Segmentation	K5 – Evaluate	6
Q4	Virtual memory and demand paging	K5 – Evaluate	8
Q5	Page-replacement techniques	K5 – Evaluate	8
Q6	Disk management	K5 – Evaluate	4
Q7	Memory-management plan from first principles	K5 – Evaluate	10
	TOTAL		50

Submission Requirements

GitHub repository link (mandatory) containing the executed code for every task, including every completed "Complete the Code" program, script and configuration file, with input files, output screenshots and a README.md.

- Your report containing the required evidence for Q1–Q7
- C source code for all programs
- Address-translation workings, graphs, measurements and command outputs

Assessment Verification

Students may be asked to explain, execute or modify any part of their design, code, calculations or results during a brief verification discussion/viva. The student should be able to explain and defend the work submitted as their own.

TABLE OF CONTENTS

Q1 – Linking, Loading and Address Binding

Q2 – Paging

Q3 – Segmentation

Q4 – Virtual Memory and Demand Paging

Q5 – Page-Replacement Techniques

Q6 – Disk Management

Q7 – Memory-Management Plan from First Principles

Overall Conclusion

References

Q1 – LINKING, LOADING AND ADDRESS BINDING (6 MARKS)

1. Introduction

A banking application is built from several modules and libraries. When an encryption library receives a security update, static linking may require distributing a newly built executable to every branch. Dynamic linking allows a program to use a shared library, which can be updated separately when compatibility is maintained.

2. Linking and Loading

The linker combines object files and libraries, resolves external symbols and produces an executable. The loader is part of the operating system. It creates the process address space, maps the executable and required shared libraries, performs required relocations and transfers control to the program entry point.

3. Address Binding

Binding type	Meaning	Evaluation
Compile-time	Addresses are fixed when the program is compiled, when the final memory location is known.	Fast address use, but relocating the program may require recompilation.
Load-time	The compiler produces relocatable code; the loader adjusts addresses when the program is loaded.	Allows the program to be placed at different memory locations.
Execution-time	Address translation is performed while the program runs, normally with hardware support such as the MMU.	Most flexible; supports virtual memory and process relocation.

4. Static Linking and Dynamic Linking

Factor	Static linking	Dynamic linking
Executable size	Usually larger because required library code is included.	Usually smaller because library code is stored separately.
Memory use	Separate executable copies can contain repeated library code.	Read-only shared library pages may be shared by processes.
Start-up	May avoid resolving external shared libraries at start-up.	The loader must map and resolve required shared libraries.

Updating	A library fix commonly requires rebuilding and redistributing the executable.	A compatible shared library can be updated separately.
Dependency	Fewer run-time library dependencies.	Program depends on the required shared library being available.

5. Position-Independent Code (PIC)

Position-independent code can execute correctly regardless of the address at which it is loaded. Shared libraries commonly use PIC so that one library can be mapped at different virtual addresses in different processes without rewriting every instruction.

6. Practical Linux Commands

```
# Build a shared library gcc -fPIC -c encrypt.c -o
encrypt.o gcc -shared -o libencrypt.so encrypt.o

# Build a dynamically linked application
gcc bank.c -L. -lencrypt -Wl,-rpath,'$ORIGIN' -o bank_dynamic

# Build a statically linked application (requires static library) gcc bank.c ./libencrypt.a -o
bank_static

# Compare executable sizes and dependencies ls -lh
bank_dynamic bank_static ldd bank_dynamic

# Inspect mapped executable and library regions cat /proc/$(pidof
bank_dynamic)/maps
```

If a static library is not available on the Linux installation, the static build command may fail. Record the exact error and install the relevant development package or state that the environment does not provide the static archive.

7. Run-Time Loading with dlopen() and dlsym()

```
#include <stdio.h>
#include <dlfcn.h>

int main(void)
{
   void *handle = dlopen("./libencrypt.so", RTLD_NOW);     if (!handle) {
      fprintf(stderr, "%s\n", dlerror());         return 1;
   }
   dlerror(); /* clear previous error */     void
(*encrypt_message)(void) =
      (void (*)(void)) dlsym(handle, "encrypt_message");

   const char *error = dlerror();     if (error !=
NULL) {
```